UDP CLIENT–SERVER ECHO PROGRAM

Linux / Ubuntu Socket Programming Lab

Objective: Create a UDP server that receives a message from a client and sends the same message back.

Requirements: Linux/Ubuntu, GCC and two terminal windows. Both programs use localhost (127.0.0.1), port 8080.

Communication: Client sends a UDP datagram → Server receives it → Server sends it back → Client displays the response.

1. UDP Server Program

Create **udp_server.c**:

```
// udp_server.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int server_fd;
    struct sockaddr_in server_addr, client_addr;
    char buffer[BUFFER_SIZE];
    socklen_t client_len;
    ssize_t n;

    server_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (server_fd < 0) { perror("socket"); return 1; }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("bind"); close(server_fd); return 1;
    }

    printf("UDP Echo Server waiting on port %d...\\n", PORT);

    while (1) {
        client_len = sizeof(client_addr);
        n = recvfrom(server_fd, buffer, sizeof(buffer), 0,
                     (struct sockaddr *)&client_addr, &client_len);

        if (n < 0) { perror("recvfrom"); continue; }

        printf("Received: %.*s\\n", (int)n, buffer);

        if (sendto(server_fd, buffer, (size_t)n, 0,
                   (struct sockaddr *)&client_addr, client_len) < 0)
            perror("sendto");
    }

    close(server_fd);
    return 0;
}
```

2. UDP Client Program

Create **udp_client.c**:

```
// udp_client.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int sock_fd;
    struct sockaddr_in server_addr;
    char message[BUFFER_SIZE], buffer[BUFFER_SIZE];
    socklen_t server_len;
    ssize_t n;

    sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock_fd < 0) { perror("socket"); return 1; }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    printf("UDP Client started.\\nEnter a message: ");
    if (fgets(message, sizeof(message), stdin) == NULL) {
        close(sock_fd); return 1;
    }

    if (sendto(sock_fd, message, strlen(message), 0,
               (struct sockaddr *)&server_addr,
               sizeof(server_addr)) < 0) {
        perror("sendto"); close(sock_fd); return 1;
    }

    server_len = sizeof(server_addr);
    n = recvfrom(sock_fd, buffer, sizeof(buffer), 0,
                 (struct sockaddr *)&server_addr, &server_len);

    if (n < 0) {
        perror("recvfrom"); close(sock_fd); return 1;
    }

    printf("Echo from server: ");
    fwrite(buffer, 1, (size_t)n, stdout);

    close(sock_fd);
    return 0;
}
```

3. Compile the Programs

```
gcc udp_server.c -o udp_server\ngcc udp_client.c -o udp_client
```

4. Execute the Lab

Terminal 1 – Start the server

```
./udp_server\nUDP Echo Server waiting on port 8080...
```

Terminal 2 – Start the client

```
./udp_client\nUDP Client started.\nEnter a message: Hello Server\nEcho from server: Hello Server
```

Expected server output

```
UDP Echo Server waiting on port 8080...\nReceived: Hello Server
```

5. Important UDP Socket Functions

Function	Purpose
socket()	Creates a UDP socket using SOCK_DGRAM.
bind()	Assigns the server IP address and port.
sendto()	Sends a datagram to a specified destination.
recvfrom()	Receives a datagram and identifies its sender.
close()	Closes the socket descriptor.

6. TCP vs UDP Comparison

Feature	TCP	UDP
Communication	Connection-oriented	Connectionless
Socket type	SOCK_STREAM	SOCK_DGRAM
Setup	Connection establishment	No connection setup
Data transfer	Byte stream	Datagrams
Reliability	Reliable, ordered delivery	No delivery guarantee
Data boundaries	Not preserved	Preserved per datagram

7. Student Experiments

1. Change the server port to 9090 in both programs and test again.
2. Send different messages and observe the server output.
3. Send a message containing approximately 500 characters.
4. Start the client before the server. Observe what happens and explain why the client may wait.
5. Run multiple clients and observe how the server handles their datagrams.
6. Modify the server to display the client's IP address and port.

8. Lab Questions

1. Why is SOCK_DGRAM used for UDP?
2. Why is listen() not required in UDP?
3. What is the purpose of recvfrom()?
4. Why does sendto() require a destination address?
5. Can UDP lose packets or deliver them out of order?
6. What happens if multiple clients send datagrams to the same server?

9. Key Learning Points

Connectionless communication, UDP datagrams, IP addresses, port numbers, socket creation, bind(), sendto(), recvfrom() and socket closure.

Important note: UDP does not guarantee delivery, ordering or retransmission. If the server is not running, the client may wait indefinitely in recvfrom() because this example has no receive timeout. UDP preserves datagram boundaries, unlike TCP's byte stream.